

Machine Learning с нуля

Неделя 3 - Git + первая ML-модель

Формат 1-2 часа в день (можно больше). Каждый день: понять -> посмотреть/почитать -> сделать -> закоммитить -> запустить.	Инструменты Python 3.10+ • Jupyter/Colab • pandas • matplotlib • scikit-learn • Git • GitHub.
---	---

Зачем в третью неделю добавлять Git?

Потому что ML - это не только ноутбук, который работает один раз. Нужно уметь сохранять историю решений, показывать прогресс, возвращаться к прошлым версиям и оформлять результат так, чтобы другой человек мог открыть проект и понять, что сделано.

Главное правило недели: всё, что сделано руками, попадает в GitHub

- Каждый день - минимум 1 осмысленный commit и push в удалённый репозиторий.
- В Git идут ноутбуки, код, README, отчёты, маленькие картинки/графики и настройки окружения.
- В Git не идут .venv, __pycache__, .env, токены, пароли, большие датасеты и случайные временные файлы.
- После push открой репозиторий в браузере и проверь, что файлы видны, README читается, ноутбук рендерится.

Минимум ML для этой недели:

- понимать задачу: классификация или регрессия;
- разделять признаки X и целевую переменную y ;
- делать train/test split без утечки данных;
- строить baseline, чтобы сравнивать модель не с нулём, а с простым ориентиром;
- обучать первые модели scikit-learn: LogisticRegression, DecisionTree, LinearRegression;
- считать метрики: accuracy, precision/recall/F1, confusion matrix, MAE, RMSE, R2;
- видеть overfitting/underfitting и проверять модель через cross-validation;
- оформлять результат в GitHub: README + ноутбук + краткий отчёт.

Итог за неделю: GitHub-репозиторий с папкой week03, ежедневными ноутбуками, README, requirements.txt, .gitignore, итоговым мини-проектом и коротким отчётом с выводами.

Что должно лежать на GitHub к концу недели

Файл/папка	Зачем нужен
README.md	объясняет цель проекта, структуру, запуск и итоги
requirements.txt	фиксирует библиотеки, чтобы проект можно было повторить
.gitignore	не даёт случайно загрузить мусор, venv, токены и большие данные
week03/notebooks/	ежедневные ноутбуки и финальный mini-project
week03/figures/	сохранённые графики, которые используются в отчёте
week03/reports/week03_summary.md	короткий человеческий вывод по неделе

Правило сдачи: присылается не архив с файлами, а ссылка на GitHub-репозиторий. По ссылке должно быть понятно, что делалось каждый день.

Карта недели

День	Тема	Результат дня	Что на Git
1	Git/GitHub и структура проекта	Создан репозиторий и рабочая структура	README, .gitignore, requirements.txt, initial commit
2	Постановка ML-задачи: X, y, train/test	Понимаешь признаки, target и split	day02_problem_setting.ipynb
3	EDA для ML + baseline	Есть первичный анализ и простая baseline-модель	графики + day03_eda_baseline.ipynb
4	Первая классификация	Обучена модель и посчитаны метрики качества	day04_classification.ipynb
5	Первая регрессия	Обучена регрессия и посчитаны MAE/RMSE/R2	day05_regression.ipynb
6	Валидация, overfitting и подбор параметров	Понимаешь, почему одной test-оценки мало	day06_validation_cv.ipynb
7	Итоговый мини-проект + оформление	Репозиторий готов к показу	final_project.ipynb + week03_summary.md

Дальше - подробности по каждому дню: что понять, какие материалы открыть, что сделать руками и что обязательно запустить на GitHub.

Структура репозитория

Можно работать в одном репозитории для всего курса, например ml-from-zero. Внутри недели держи отдельную папку week03.

```
ml-from-zero/  
  README.md  
  requirements.txt  
  .gitignore  
  week03/  
    notebooks/  
      day01_git_setup.ipynb  
      day02_problem_setting.ipynb  
      day03_eda_baseline.ipynb  
      day04_classification.ipynb  
      day05_regression.ipynb  
      day06_validation_cv.ipynb  
      day07_final_project.ipynb  
    src/  
      utils.py  
    figures/  
    reports/  
      week03_summary.md  
    data/  
      README.md
```

Ежедневный Git-ритуал

```
git status  
# после работы:  
git add week03 README.md requirements.txt .gitignore  
git commit -m "day3: add EDA and baseline"  
git push
```

- Перед коммитом перезапусти ноутбук: Kernel -> Restart & Run All, чтобы не отправлять битый результат.
- Сообщение коммита должно отвечать на вопрос: что изменилось?
- Если работаешь через ветки: day03-eda -> pull request -> merge в main. Если пока тяжело, коммить прямо в main, но каждый день пушь.

День 1 - Git/GitHub и структура проекта

Цель дня

Создать репозиторий, настроить базовый workflow и подготовить место, куда каждый день будет уходить работа.

Что понять

- Что такое repository, commit, branch, remote и push.
- Зачем нужны README.md, requirements.txt и .gitignore.
- Почему история коммитов - это часть результата, а не техническая мелочь.

Материалы

- Pro Git на русском: главы про основы Git и первоначальную настройку.
- GitHub Docs: Hello World - репозиторий, ветки, коммиты, pull request.
- GitHub Docs: Ignoring files - что не нужно коммитить.

Задания (сделать руками)

- Проверь установку: `git --version`. Настрой имя и email через `git config --global`.
- Создай репозиторий ml-from-zero на GitHub и клонируй его на компьютер.
- Собери структуру папок week03/notebooks, week03/src, week03/figures, week03/reports.
- Добавь README.md с целью недели и таблицей прогресса.
- Добавь .gitignore: .venv, __pycache__, .ipynb_checkpoints, .env, data/raw/.
- Добавь requirements.txt: numpy, pandas, matplotlib, scikit-learn, jupyter.

Git-результат дня

- Commit: day1: initialize week3 repository.
- Push на GitHub и проверка в браузере: README виден, структура папок видна.
- В README вставлена строка прогресса за День 1: done / ссылка на ноутбук или описание.

День 2 - Постановка ML-задачи: X, y, train/test

Цель дня

Понять, что именно модель должна предсказывать, и научиться правильно делить данные на обучение и проверку.

Что понять

- Supervised learning: модель учится по примерам, где уже есть правильный ответ.

- Классификация vs регрессия: класс/категория против числа.
- Признаки X и целевая переменная y.
- Train/test split: модель учится на train, честная проверка идёт на test.
- Data leakage: нельзя подглядывать в test во время подготовки решения.

Материалы

- scikit-learn: Getting Started.
- scikit-learn: train_test_split.
- scikit-learn tutorial: introduction to machine learning vocabulary.

Задания (сделать руками)

- Создай notebook week03/notebooks/day02_problem_setting.ipynb.
- Загрузи встроенный датасет: load_iris или load_breast_cancer для классификации.
- Выведи shape, названия признаков, target names, первые строки таблицы.
- Сформулируй 3 вопроса: что предсказываем, какие признаки есть, какая метрика будет важна.
- Раздели X и y, сделай train_test_split(test_size=0.2, random_state=42, stratify=y).
- Проверь размеры X_train, X_test, y_train, y_test и баланс классов.

Git-результат дня

- Commit: day2: define ML problem and train test split.
- В README добавлена ссылка на day02 notebook и 2-3 строки: какая задача и какой датасет.

День 3 - EDA для ML + baseline

Цель дня

Сделать первичный анализ данных и построить baseline, чтобы у модели был честный ориентир для сравнения.

Что понять

- EDA перед моделью: пропуски, типы, распределения, выбросы, баланс классов.
- Baseline - простое решение, которое показывает минимальную планку качества.
- DummyClassifier/DummyRegressor или ручной baseline: самый частый класс / среднее значение.

Материалы

- scikit-learn: model evaluation и dummy estimators в разделе Model selection and evaluation.
- Документация pandas/matplotlib из прошлой недели для таблиц и графиков.

- Примерные идеи графиков: histogram, bar chart, scatter plot, correlation heatmap через pandas/matplotlib.

Задания (сделать руками)

- Создай notebook day03_eda_baseline.ipynb.
- Сделай info/describe, проверь пропуски, типы и распределение target.
- Построй 3 графика и сохрани их в week03/figures/.
- Сделай baseline: для классификации DummyClassifier(strategy="most_frequent"), для регрессии DummyRegressor(strategy="mean").
- Посчитай baseline-метрику и напиши 5-7 строк выводов: что в данных важно для модели.

Git-результат дня

- Commit: day3: add EDA and baseline.
- На GitHub должны быть notebook, сохранённые графики и обновлённый README с baseline-результатом.

День 4 - Первая классификация

Цель дня

Обучить первую классификационную модель, сравнить её с baseline и понять, что означают метрики.

Что понять

- fit/predict: как scikit-learn обучает модель и делает прогноз.
- Accurasy не всегда достаточно: precision, recall, F1 полезны при дисбалансе классов.
- Confusion matrix показывает, где модель путается.
- Модель надо сравнивать с baseline, а не просто радоваться высокому числу.

Материалы

- scikit-learn: supervised learning.
- scikit-learn: model evaluation metrics.
- scikit-learn: LogisticRegression или DecisionTreeClassifier docs при необходимости.

Задания (сделать руками)

- Создай notebook day04_classification.ipynb.
- На датасете Iris или Breast Cancer обучи LogisticRegression и DecisionTreeClassifier.
- Сделай predict на test и посчитай accuracy, classification_report, confusion_matrix.
- Сравни модели между собой и с baseline из Дня 3.

- Напиши 6-8 строк выводов: какая модель лучше, где ошибается, можно ли доверять результату.

Git-результат дня

- Commit: day4: train first classification models.
- В README добавлена маленькая таблица: baseline vs model 1 vs model 2.

День 5 - Первая регрессия

Цель дня

Понять отличие регрессии от классификации и обучить модель, которая предсказывает число.

Что понять

- Regression target - число, поэтому метрики другие.
- MAE - средняя абсолютная ошибка, RMSE сильнее штрафует большие ошибки, R2 показывает долю объяснённой вариации.
- График y_{true} vs y_{pred} помогает увидеть, насколько прогнозы похожи на реальные значения.

Материалы

- scikit-learn: regression metrics в Model evaluation.
- scikit-learn: LinearRegression и DecisionTreeRegressor.
- scikit-learn datasets: load_diabetes как простой встроенный регрессионный пример.

Задания (сделать руками)

- Создай notebook day05_regression.ipynb.
- Загрузи load_diabetes, собери X и y, сделай train/test split.
- Обучи DummyRegressor, LinearRegression и DecisionTreeRegressor.
- Посчитай MAE, RMSE и R2 для каждой модели.
- Построй график y_{true} vs y_{pred} для лучшей модели.
- Напиши 6-8 строк выводов: насколько ошибка большая и что можно улучшать.

Git-результат дня

- Commit: day5: train first regression models.
- На GitHub есть notebook, график и обновлённый README с метриками регрессии.

День 6 - Валидация, overfitting и подбор параметров

Цель дня

Увидеть, что модель может хорошо помнить train, но плохо работать на новых данных, и научиться проверять её устойчивее.

Что понять

- Overfitting: модель слишком хорошо подстроилась под train и хуже обобщает.
- Underfitting: модель слишком простая и плохо учится даже на train.
- Cross-validation оценивает модель на нескольких разбиениях данных.
- Гиперпараметр - настройка модели до обучения, например max_depth у дерева.

Материалы

- scikit-learn: Cross-validation.
- scikit-learn: Tuning the hyper-parameters of an estimator.
- scikit-learn: DecisionTreeClassifier/Regressor параметры max_depth, min_samples_leaf.

Задания (сделать руками)

- Создай notebook day06_validation_cv.ipynb.
- Возьми DecisionTreeClassifier или DecisionTreeRegressor и проверь max_depth = 1, 2, 3, 5, None.
- Для каждого max_depth сравни train score и test score.
- Сделай cross_val_score на 5 фолдах и посчитай среднее/стандартное отклонение.
- Построй простой line plot: max_depth -> score.
- Напиши вывод: где underfitting, где возможный overfitting, какую настройку выбрал бы сейчас.

Git-результат дня

- Commit: day6: add validation and cross validation.
- В README добавлен вывод: почему выбранный параметр выглядит лучше.

День 7 - Итоговый мини-проект + оформление GitHub

Цель дня

Собрать одну аккуратную работу, которую можно отправить наставнику/работодателю/другу: открыть репозиторий и понять ход решения.

Что понять

- Мини-проект - это не только код, но и понятная история: задача -> данные -> baseline -> модель -> метрики -> выводы.
- README должен объяснять, как запустить проект и где смотреть результат.
- Хороший финал недели - воспроизводимый ноутбук: все ячейки идут сверху вниз без ручных правок.

Материалы

- GitHub Docs: repositories, branches, commits, pull requests.
- scikit-learn Getting Started и Model evaluation для проверки терминов.
- Свои ноутбуки дней 2-6 как база для финального проекта.

Задания (сделать руками)

- Выбери один основной датасет: классификация или регрессия.
- Создай day07_final_project.ipynb: загрузка данных, EDA, split, baseline, 2 модели, метрики, выводы.
- Сделай reports/week03_summary.md: 12-20 строк о задаче, данных, метриках и результатах.
- Обнови README: цель недели, структура проекта, как установить зависимости, как запустить notebook, итоговая таблица метрик.
- Проверь GitHub: все ссылки открываются, notebook рендерится, нет лишних файлов и секретов.
- Опционально: создай branch week03-final и pull request в main, даже если работаешь один.

Git-результат дня

- Commit: day7: finish week3 mini project.
- Final push. В README есть итог недели и ссылка/путь к финальному notebook.
- Definition of Done: репозиторий можно открыть с нуля и понять, что сделано за неделю.

Чек-лист перед сдачей недели

- GitHub-репозиторий доступен по ссылке (публичный или приватный с доступом наставнику).
- Есть минимум 7 коммитов за неделю, лучше - по 1-3 коммита в день.
- README.md объясняет: цель, структура, как запустить, где итоговый notebook, какие метрики получились.
- requirements.txt содержит используемые библиотеки.
- .gitignore защищает от мусора: .venv, __pycache__, .ipynb_checkpoints, .env, data/raw/.
- Все ноутбуки перезапущены сверху вниз без ошибок.
- В репозитории нет паролей, токенов, личных файлов и больших датасетов.
- В reports/week03_summary.md есть короткий человеческий вывод: что получилось, что не получилось, что улучшить дальше.

Шаблон README для GitHub

```
# ML с нуля - Неделя 3

## Цель
Научиться вести ML-проект в GitHub и обучить первые модели scikit-learn.

## Структура
- week03/notebooks/ - ежедневные ноутбуки
- week03/src/ - вспомогательные функции
- week03/figures/ - графики
- week03/reports/ - краткие выводы

## Как запустить
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
jupyter notebook

## Итоги
| День | Статус | Результат |
| --- | --- | --- |
| 1 | done | repo setup |
| 2 | done | train/test split |
| 3 | done | EDA + baseline |
| 4 | done | classification |
| 5 | done | regression |
| 6 | done | cross-validation |
| 7 | done | final project |
```

Как писать commit messages

- Плохо: update, fix, final, notebook.
- Хорошо: day3: add class balance plot, day4: compare logistic regression and tree, day6: add cross validation table. Один логический шаг - один commit.

Список источников

[1] Pro Git, русская версия

<https://git-scm.com/book/ru/v2>

[2] GitHub Docs: Hello World

<https://docs.github.com/en/get-started/start-your-journey/hello-world>

[3] GitHub Docs: Quickstart for repositories

<https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories>

[4] GitHub Docs: Ignoring files

<https://docs.github.com/en/get-started/git-basics/ignoring-files>

[5] GitHub Docs: About branches

<https://docs.github.com/articles/about-branches>

[6] scikit-learn: Getting Started

https://scikit-learn.org/stable/getting_started.html

[7] scikit-learn: train_test_split

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

[8] scikit-learn: Supervised learning

https://scikit-learn.org/stable/supervised_learning.html

[9] scikit-learn: Model evaluation metrics

https://scikit-learn.org/stable/modules/model_evaluation.html

[10] scikit-learn: Cross-validation

https://scikit-learn.org/stable/modules/cross_validation.html

Куда двигаться после Недели 3

- Повторить финальный мини-проект на другом датасете и сравнить результаты.
- Добавить preprocessing pipeline: StandardScaler, OneHotEncoder, ColumnTransformer.
- Начать Неделю 4: preprocessing, feature engineering, pipelines и более аккуратная оценка моделей.